

Resumo de Estudo — Aula 11

Construção de GLCs e Ambiguidade • Seção 2.1

Na aula anterior aprendemos o que é uma gramática livre-do-contexto e como ela gera cadeias. Agora vamos aprender o lado prático — **como projetar** uma GLC para uma linguagem dada — e um problema importante que surge nesse processo: a **ambiguidade**, quando uma mesma cadeia pode ter mais de uma estrutura.

1. Projetando gramáticas livres-do-contexto

Projetar uma GLC exige certa criatividade, mas existem estratégias que quase sempre funcionam. A dica geral para testar uma gramática que você propôs é **simular a derivação de algumas cadeias** e verificar se batem com a linguagem desejada. Vamos primeiro construir gramáticas para linguagens simples (com $\Sigma = \{a, b\}$) e depois extrair as estratégias gerais.

Bloco de gramáticas básicas (memorize os padrões)

Estas são as "peças de LEGO" a partir das quais montamos gramáticas maiores:

- | | | | |
|----|-----------------------------|--|--|
| 1. | $L = \{a\}$ | $S \rightarrow a$ | |
| 2. | $L = \{\epsilon\}$ | $S \rightarrow \epsilon$ | |
| 3. | $L = \{a, b\}$ | $S \rightarrow a \mid b$ | |
| 4. | $L = a^*$ | $S \rightarrow aS \mid \epsilon$ | (ou $S \rightarrow Sa \mid \epsilon$) |
| 5. | $L = a^+$ | $S \rightarrow aS \mid a$ | (ou $S \rightarrow Sa \mid a$) |
| 6. | $L = \Sigma^* = \{a, b\}^*$ | $S \rightarrow aS \mid bS \mid \epsilon$ | |
| 7. | $L = \Sigma^+ = \{a, b\}^+$ | $S \rightarrow aS \mid bS \mid a \mid b$ | |

Observe o par de opções em 4 e 5: pôr a variável à direita (aS) constrói a cadeia da esquerda para a direita; pô-la à esquerda (Sa) constrói da direita para a esquerda. Ambas geram a mesma linguagem.

Linguagens um pouco mais elaboradas

$L = \{ \text{cadeias que começam e terminam com } a \}$

A expressão regular é $a\Sigma^*a \cup \{a\}$. A parte $a\Sigma^*a$ é um "aninhamento": um a fixo no início, um a fixo no fim, e qualquer coisa no meio (uma sub-gramática T que gera Σ^*). Adicionamos a regra $S \rightarrow a$ para cobrir a cadeia unitária «a»:

$$\begin{aligned} S &\rightarrow a T a \mid a \\ T &\rightarrow Ta \mid Tb \mid \epsilon \end{aligned}$$

$L = \{ \text{cadeias que começam e terminam com o mesmo símbolo} \}$

Aqui $L = L_1 \cup L_2$, onde L_1 começa/termina com a e L_2 com b. A estratégia da **união** é criar uma variável inicial que escolhe entre as duas:

$$\begin{aligned} S &\rightarrow S1 \mid S2 \\ S1 &\rightarrow a \mid a T a \\ S2 &\rightarrow b \mid b T b \\ T &\rightarrow Ta \mid Tb \mid \epsilon \end{aligned}$$

$L = \{ a^n b^n \mid n \geq 0 \}$ — a estratégia recursiva

Toda palavra não-vazia se escreve em função de uma menor: $w_n = a \cdot w_{n-1} \cdot b$, com caso base $w_0 = \epsilon$. Essa recursão vira diretamente uma regra que envolve a variável de si mesma, cercada pelos símbolos casados:

$$S \rightarrow a S b \mid \epsilon$$

$L = \{ a^m b^m c^n d^n \mid m, n \geq 0 \}$ — concatenação

$L = L_1 \cdot L_2$ (concatenação), com $L_1 = \{a^m b^m\}$ e $L_2 = \{c^n d^n\}$. Já sabemos construir cada uma; para concatenar, basta uma variável inicial que gera uma seguida da outra:

$$\begin{aligned} S &\rightarrow S1 S2 \\ S1 &\rightarrow a S1 b \mid \epsilon \\ S2 &\rightarrow c S2 d \mid \epsilon \end{aligned}$$

2. As quatro estratégias gerais

Generalizando os exemplos acima, temos um conjunto de técnicas que cobrem a maioria dos casos de projeto de GLCs:

Estratégia 1 — Composição de linguagens mais simples

LLCs costumam ser combinações de LLCs menores. Para a **união** $L_1 \cup L_2$: construa G_1 (variável inicial S_1) e G_2 (variável inicial S_2) separadamente, junte todas as regras e adicione $S \rightarrow S1 \mid S2$. Para a **concatenação** $L_1 \cdot L_2$: adicione $S \rightarrow S1 S2$.

Estratégia 2 — Toda linguagem regular tem uma GLC

Se você consegue um AFD/AFN ou uma expressão regular para a linguagem, a conversão para GLC é mecânica.

Convertendo um autômato (AFD/AFN) em GLC:

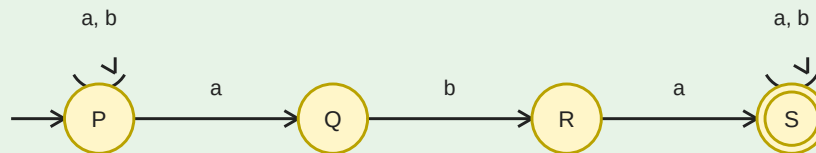
- Crie uma variável R_i para cada estado q_i .
- Para cada transição $q_i \xrightarrow{(a)} q_j$, adicione $R_i \rightarrow a R_j$; para transição- ϵ $q_i \xrightarrow{(\epsilon)} q_j$, adicione $R_i \rightarrow R_j$.
- Para cada estado de aceitação q_i , adicione $R_i \rightarrow \epsilon$.
- Torne R_0 (do estado inicial q_0) a variável inicial.

Convertendo uma expressão regular em GLC (recursivamente sobre a estrutura da ER):

$R = a$	$\rightarrow$	$S \rightarrow a$	(idem para ϵ)
$R = R1 \cup R2$	$\rightarrow$	$S \rightarrow S1 \mid S2$	
$R = R1 \cdot R2$	$\rightarrow$	$S \rightarrow S1 S2$	
$R = R1^*$	$\rightarrow$	$S \rightarrow S1 S \mid \epsilon$	
$R = R1^+$	$\rightarrow$	$S \rightarrow S1 S \mid S1$	

Aplicação — $L = \{ w \in \{a,b\}^* \mid w \text{ contém a subcadeia aba} \}$

Primeiro projetamos o autômato (AFN) que reconhece a linguagem: ele fica "parado" em P consumindo a's e b's, avança para Q ao ver o «a», para R ao ver «b», para S ao ver «a» (fechando o «aba»), e então aceita qualquer sufixo:



Aplicando a conversão estado $\rightarrow$ variável (P,Q,R,S viram variáveis; loops e transições viram regras; o estado de aceitação S ganha $S \rightarrow \epsilon$):

$$\begin{aligned} P &\rightarrow aP \mid bP \mid aQ \\ Q &\rightarrow bR \\ R &\rightarrow aS \\ S &\rightarrow aS \mid bS \mid \epsilon \end{aligned}$$

Note a correspondência direta: cada auto-loop de P vira $P \rightarrow aP$ e $P \rightarrow bP$; a transição $P \xrightarrow{a} Q$ vira $P \rightarrow aQ$; o estado final S produz a regra $S \rightarrow \epsilon$. (Há uma versão mais compacta via expressão regular, abaixo.)

Alternativamente, partindo da expressão regular $\Sigma^*aba\Sigma^*$, a conversão dá uma GLC bem mais enxuta para a mesma linguagem:

$$\begin{aligned} S &\rightarrow T a b a T \\ T &\rightarrow aT \mid bT \mid \epsilon \end{aligned}$$

Estratégia 3 — Subcadeias "ligadas" (correspondência)

Quando duas partes da cadeia precisam corresponder em quantidade ilimitada (uma máquina teria de "lembrar" quanto viu de uma para conferir a outra), use uma regra recursiva da forma $R \rightarrow u R v$. Ela gera camadas casadas de u e v de fora para dentro. Foi o que fizemos em $a^n b^n$ com $S \rightarrow aSb \mid \epsilon$.

Estratégia 4 — Estruturas recursivas aninhadas

Em linguagens complexas, uma estrutura pode reaparecer dentro de outra (ou de si mesma). Ex.: em expressões aritméticas, no lugar de um operando a pode surgir uma expressão parentizada inteira. Para conseguir isso, coloque a variável que gera a estrutura exatamente na posição onde ela pode reaparecer — como o <EXPR> dentro de <FATOR> $\rightarrow (\text{<EXPR>})$ na gramática G4.

3. Ambiguidade

Às vezes uma gramática gera a mesma cadeia de **várias maneiras estruturalmente diferentes** — ou seja, com árvores sintáticas distintas. Nesse caso dizemos que a cadeia é **derivada ambigualmente**, e que a gramática é **ambígua**. Como cada árvore corresponde a uma interpretação (a um "significado"), a ambiguidade é indesejável em linguagens de programação, onde um programa deve ter uma única leitura.

Definição precisa (cuidado com a pegadinha!)

Uma cadeia é gerada ambigualmente quando tem **duas árvores sintáticas diferentes** — **não** quando tem duas derivações diferentes. Duas derivações podem diferir só pela *ordem* em que substituem as variáveis, sem mudar a estrutura. Para eliminar essa distinção irrelevante, definimos um tipo padronizado de derivação (veja abaixo).

Derivação mais à esquerda

Uma **derivação mais à esquerda** é aquela em que, a cada passo, substituímos sempre a variável **mais à esquerda** da cadeia intermediária. Fixando essa ordem, cada árvore sintática corresponde a exatamente uma derivação mais à esquerda — então podemos comparar árvores comparando derivações mais à esquerda. Exemplo com G4, derivando $(a + a) \times a$:

$$\begin{aligned} E &\Rightarrow T \Rightarrow T \times F \Rightarrow F \times F \Rightarrow (E) \times F \Rightarrow (E + T) \times F \Rightarrow (T + T) \times F \\ &\Rightarrow (F + T) \times F \Rightarrow (a + T) \times F \Rightarrow (a + F) \times F \Rightarrow (a + a) \times F \Rightarrow (a + a) \times a \end{aligned}$$

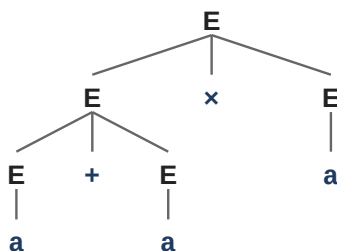
Exemplo clássico: G4 (não-ambígua) versus G5 (ambígua)

A gramática G4 da aula passada captura a precedência de $\times$ sobre $+$, e por isso cada expressão tem uma **única** árvore — logo, um único resultado. Já a gramática G5, abaixo, gera as mesmas expressões, porém de forma ambígua:

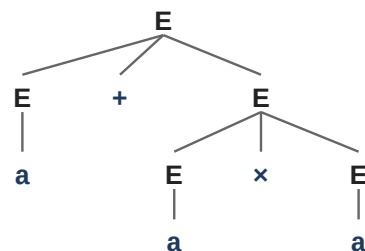
Gramática G5 (ambígua)

$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a$$

A cadeia $a + a \times a$ tem duas árvores em G5. À esquerda, a estrutura agrupa $(a+a)$ primeiro e multiplica por a ; à direita, agrupa $(a \times a)$ primeiro e soma a . Se $a = 3$, a árvore da esquerda dá **18** e a da direita dá **12**:



Esquerda: $(a+a) \times a \rightarrow \text{resultado } 18$



Direita: $a + (a \times a) \rightarrow \text{resultado } 12$

Ambiguidade inerente

Muitas vezes uma gramática ambígua pode ser reescrita como uma gramática não-ambígua equivalente (foi o que G4 fez em relação a G5). Mas existem linguagens que **só** podem ser geradas por gramáticas ambíguas — são as **inerentemente ambíguas**. Exemplo:

$$L = \{ a^i b^j c^k \mid i = j \text{ ou } j = k, \quad i, j, k \geq 0 \}$$

Para a cadeia aaabbbccc não existe gramática com derivação única, porque ela satisfaz *ao mesmo tempo* $i = j$ e $j = k$, e qualquer gramática precisa "decidir" por qual das duas condições gerá-la. Note que L é a união de $\{a^i b^i c^*\}$ com $\{a^* b^j c^j\}$.

Ambiguidade em linguagens de programação: o problema do «dangling else»

Um caso famoso surge com a regra de comandos condicionais:

```
Statement → if Cond then Statement
           | if Cond then Statement else Statement
           | ...
```

A sentença «if a then if b then s1 else s2» pode ser lida de duas formas — o «else» pode pertencer ao primeiro ou ao segundo «if»:

- (1) if a then { if b then s1 } else s2
- (2) if a then { if b then s1 else s2 }

Formas de resolver: (a) **modificar a gramática** para torná-la não-ambígua — por exemplo, tornando um delimitador como «endif»/«fi» obrigatório; ou (b) manter a gramática e adotar uma **regra de desambiguação** — a mais comum é associar o «else» sempre ao «if» mais próximo (o último encontrado).

Exercícios resolvidos

Abaixo resolvemos os exercícios propostos nos slides. Onde havia apenas o enunciado, apresentamos a gramática e justificamos.

Exercício 2.4 — Dê GLCs para as linguagens abaixo ($\Sigma = \{0, 1\}$)

(a) $\{ w \mid w \text{ contém pelo menos três 1s} \}$

Basta forçar três 1s obrigatórios, separados por "lixo" arbitrário gerado por $X = \Sigma^*$:

$$\begin{aligned} S &\rightarrow X 1 X 1 X 1 X \\ X &\rightarrow 0X \mid 1X \mid \epsilon \end{aligned}$$

(b) $\{ w \mid w \text{ começa e termina com o mesmo símbolo} \}$

Mesma ideia da união vista na aula (adaptada a $\{0,1\}$); inclui as cadeias unitárias 0 e 1:

$$\begin{aligned} S &\rightarrow 0 T 0 \mid 1 T 1 \mid 0 \mid 1 \\ T &\rightarrow 0T \mid 1T \mid \epsilon \end{aligned}$$

(c) $\{ w \mid \text{o comprimento de } w \text{ é ímpar} \}$

Ideia: um símbolo central obrigatório (comprimento 1), e a possibilidade de acrescentar **dois** símbolos por vez à frente, o que mantém a paridade ímpar:

$$S \rightarrow 0 \mid 1 \mid 00S \mid 01S \mid 10S \mid 11S$$

Cada regra com dois símbolos à frente soma 2 ao comprimento (mantém ímpar); os casos base 0 e 1 dão comprimento 1. ✓

(d) $\{ w \mid \text{comprimento ímpar e o símbolo central é } 0 \}$

A variável S constrói pares casados de símbolos quaisquer em torno de um centro; o centro é forçado a ser 0:

$$S \rightarrow 0 S 0 \mid 0 S 1 \mid 1 S 0 \mid 1 S 1 \mid 0$$

A regra recursiva acrescenta um símbolo de cada lado (mantém o centro no meio) e o caso base $S \rightarrow 0$ fixa o símbolo central. ✓

(e) $\{ w \mid w = w^R, \text{ isto é, } w \text{ é um palíndromo} \}$

Um palíndromo se constrói de fora para dentro, adicionando o mesmo símbolo nas duas pontas; os casos base cobrem comprimento par (ϵ) e ímpar (0 ou 1):

$$S \rightarrow 0 S 0 \mid 1 S 1 \mid 0 \mid 1 \mid \epsilon$$

(f) O conjunto vazio

Uma gramática cuja variável inicial não pode terminar nenhuma derivação (nunca chega a uma cadeia só de terminais) gera a linguagem vazia:

$$S \rightarrow S$$

Como S só pode ser trocado por S, nenhuma cadeia de terminais é jamais obtida, logo $L(G) = \emptyset$. ✓

Outros exercícios da aula

Adapte $S \rightarrow aSb \mid \epsilon$ para $L = \{ a^n b^{2n} \mid n \geq 0 \}$

Agora cada a precisa corresponder a **dois** b. Basta acrescentar dois b por aplicação da regra:

$$S \rightarrow a S b b \mid \epsilon$$

Derivando: $S \Rightarrow aSbb \Rightarrow aaSbbbb \Rightarrow aabbbb$ ($n=2$): a^2b^4 . ✓

Gramática para a linguagem inerentemente ambígua $\{ a^i b^j c^k \mid i=j \text{ ou } j=k \}$

Usando $L = \{ a^i b^j c^k \} \cup \{ a^* b^j c^j \}$, construímos uma sub-gramática para cada parte e unimos:

$$\begin{aligned} S &\rightarrow L1 \mid L2 \\ L1 &\rightarrow X C && (a^i b^i \text{ seguido de } c's) \\ X &\rightarrow a X b \mid \epsilon \\ C &\rightarrow c C \mid \epsilon \\ L2 &\rightarrow A Y && (a's \text{ seguido de } b^j c^j) \\ A &\rightarrow a A \mid \epsilon \\ Y &\rightarrow b Y c \mid \epsilon \end{aligned}$$

A cadeia aaabbbccc pode ser gerada tanto pelo ramo L1 ($i=j=3$, com C gerando ccc) quanto pelo ramo L2 ($j=k=3$, com A gerando aaa) — é exatamente esse duplo caminho que caracteriza a ambiguidade inerente. ✓

Mostre que «the girl touches the boy with the flower» é ambígua em G2

A ambiguidade está em **a quem o «with the flower» se liga**. Há duas leituras:

- **Leitura 1** — a flor é o *instrumento*: a garota, usando a flor, toca o menino. O «with the flower» (<PREP-PHRASE>) anexa-se ao <VERB-PHRASE> (junto do verbo «touches»).
- **Leitura 2** — a flor está *com o menino*: a garota toca o menino que está com a flor. O «with the flower» anexa-se ao <NOUN-PHRASE> «the boy».

As duas leituras correspondem a duas árvores sintáticas distintas (o <PREP-PHRASE> pendura em pontos diferentes da árvore), o que prova que G2 é ambígua. ✓